

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №4
Вариант №320801

Группа Р3208

Студент Горин Семён Дмитриевич

Преподаватель Наумова Надежда Александровна

Содержание

1. Текст задания	3
2. Листинг MBean'ов	3
3. Показания JConsole	7
4. Показания VisualVM	9
5. Устранение проблем с производительностью в заданной программе	11
6. Вывод	13

1. Текст задания

Вариант 320801

Внимание! У разных вариантов разный текст задания!

1. Для своей программы из лабораторной работы #3 по дисциплине "Веб-программирование" реализовать:

- MBean, считающий общее число установленных пользователем точек, а также число точек, не попадающих в область. В случае, если координаты установленной пользователем точки вышли за пределы отображаемой области координатной плоскости, разработанный MBean должен отправлять оповещение об этом событии.
- MBean, определяющий процентное отношение "промахов" к общему числу кликов пользователя по координатной плоскости.

2. С помощью утилиты JConsole провести мониторинг программы:

- Снять показания MBean-классов, разработанных в ходе выполнения задания 1.
- Определить количество классов, загруженных в JVM в процессе выполнения программы.

3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- Снять график изменения показаний MBean-классов, разработанных в ходе выполнения задания 1, с течением времени.
- Определить имя класса, объекты которого занимают наибольший объем памяти JVM; определить пользовательский класс, в экземплярах которого находятся эти объекты.

4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в программе. По результатам локализации и устранения проблемы необходимо составить отчет, в котором должна содержаться следующая информация:

- Описание выявленной проблемы.
- Описание путей устранения выявленной проблемы.
- Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.

Студент должен обеспечить возможность воспроизведения процесса поиска и локализации проблемы по требованию преподавателя.

2. Листинг MBean'ов

```
1: package ru.astrosoup.weblab3.monitoring;
2:
3: import ru.astrosoup.weblab3.DT0s.hit.AreaHitResponse;
4:
5: public interface HitCheckerMBean {
6:     long getTotalHits(Long id);
7:     long getMissedHits(Long id);
8:     long getTotalHitsAllUsers();
9:     long getMissedHitsAllUsers();
10:    void addPoint(AreaHitResponse hit, Long id);
11: }
12:
```

Листинг 1 — Интерфейс первого Mbean

```
1: package ru.astrosoup.weblab3.monitoring;
2:
3: import jakarta.annotation.PostConstruct;
4: import jakarta.annotation.PreDestroy;
5: import jakarta.ejb.*;
6: import ru.astrosoup.weblab3.DT0s.hit.AreaHitResponse;
7:
8: import javax.management.*;
9: import java.lang.management.ManagementFactory;
10: import java.util.HashMap;
11: import java.util.Map;
12: import java.util.concurrent.atomic.AtomicLong;
13: import java.util.logging.Logger;
14: @Startup
15: @Singleton
16: @ConcurrencyManagement(ConcurrencyManagementType.CONTAINER)
17: public class HitChecker extends NotificationBroadcasterSupport
18:     implements HitCheckerMBean {
19:     private static final Logger log =
20:         Logger.getLogger(HitChecker.class.getName());
```

```

20:
21:     private final Map<Long, Long> totalPointsById = new HashMap<>();
22:     private final Map<Long, Long> missedPointsById = new HashMap<>();
23:     private final AtomicLong sequenceNumber = new AtomicLong(1L);
24:
25:     @PostConstruct
26:     public void register() {
27:         try {
28:             MBeanServer mbs =
ManagementFactory.getPlatformMBeanServer();
29:             ObjectName name = new
ObjectName("ru.astrosoup.weblab3:type=HitChecker");
30:             mbs.registerMBean(this, name);
31:             log.info(">>> HitChecker registered with JMX");
32:         } catch (Exception e) {
33:             log.severe(">>> HitChecker JMX registration failed: " +
e.getMessage());
34:         }
35:     }
36:
37:     @PreDestroy
38:     public void unregister() {
39:         try {
40:             MBeanServer mbs =
ManagementFactory.getPlatformMBeanServer();
41:             ObjectName name = new
ObjectName("ru.astrosoup.weblab3:type=HitChecker");
42:             mbs.unregisterMBean(name);
43:         } catch (Exception e) {
44:             log.severe(">>> HitChecker JMX unregistration failed: " +
e.getMessage());
45:         }
46:     }
47:
48:
49:     @Override
50:     @Lock(LockType.READ)
51:     public long getTotalHitsAllUsers() {
52:         return totalPointsById.values().stream()
53:             .mapToLong(Long::longValue)
54:             .sum();
55:     }
56:
57:     @Override
58:     @Lock(LockType.READ)
59:     public long getMissedHitsAllUsers() {
60:         return missedPointsById.values().stream()
61:             .mapToLong(Long::longValue)
62:             .sum();
63:     }
64:
65:     @Override
66:     @Lock(LockType.READ)

```

```

67:     public long getTotalHits(Long id) {
68:         return totalPointsById.getOrDefault(id, 0L);
69:     }
70:
71:     @Override
72:     @Lock(LockType.READ)
73:     public long getMissedHits(Long id) {
74:         return missedPointsById.getOrDefault(id, 0L);
75:     }
76:
77:     @Lock(LockType.WRITE)
78:     public void addPoint(AreaHitResponse hit, Long id) {
79:         totalPointsById.merge(id, 1L, Long::sum);
80:
81:         if (!hit.isHit()) {
82:             missedPointsById.merge(id, 1L, Long::sum);
83:         }
84:
85:         if (Math.abs(hit.getX()) > 5 || Math.abs(hit.getY()) > 5) {
86:             sendNotification(new Notification(
87:                 "ru.astrosoup.weblab3.monitoring.point-out-of-
bounds",
88:                 this,
89:                 sequenceNumber.getAndIncrement(),
90:                 System.currentTimeMillis(),
91:                 String.format("The hit (%d, %f) was out of bounds
for the displayed field.",
92:                             hit.getX(), hit.getY())
93:             ));
94:         }
95:     }
96:
97:     @Override
98:     public MBeanNotificationInfo[] getNotificationInfo() {
99:         return new MBeanNotificationInfo[]{
100:             new MBeanNotificationInfo(
101:                 new String[]
{ "ru.astrosoup.weblab3.monitoring.point-out-of-bounds" },
102:                 Notification.class.getName(),
103:                 "Point is outside the shown plane"
104:             )
105:         };
106:     }
107: }

```

Листинг 2 — Имплементация HitCheckerMBean

```

1: package ru.astrosoup.weblab3.monitoring;
2:
3: public interface MissedHitCalculatorMBean {
4:     double getMissedToTotalHitsInPercents(Long id);
5:     double getMissedToTotalHitsInPercentsAllUsers();
6: }
7:

```

Листинг 3 — Интерфейс второго MBean

```
1: package ru.astrosoup.weblab3.monitoring;
2:
3: import jakarta.annotation.PostConstruct;
4: import jakarta.annotation.PreDestroy;
5: import jakarta.ejb.Lock;
6: import jakarta.ejb.LockType;
7: import jakarta.ejb.Startup;
8: import jakarta.inject.Inject;
9: import jakarta.ejb.Singleton;
10:
11: import javax.management.MBeanServer;
12: import javax.management.ObjectName;
13: import java.lang.management.ManagementFactory;
14: import java.util.logging.Logger;
15: @Startup
16: @Singleton
17: public class MissedHitCalculator implements MissedHitCalculatorMBean {
18:
19:     private static final Logger log =
20:         Logger.getLogger(MissedHitCalculator.class.getName());
21:
22:     @Inject
23:     HitCheckerMBean hitChecker;
24:
25:     @PostConstruct
26:     public void register() {
27:         try {
28:             MBeanServer mbs =
29:                 ManagementFactory.getPlatformMBeanServer();
30:             ObjectName name = new
31:                 ObjectName("ru.astrosoup.weblab3:type=MissedHitCalculator");
32:             mbs.registerMBean(this, name);
33:             log.info(">>> MissedHitCalculator registered with JMX");
34:         } catch (Exception e) {
35:             log.severe(">>> MissedHitCalculator JMX registration
36: failed: " + e.getMessage());
37:         }
38:     }
39:
40:     @PreDestroy
41:     public void unregister() {
42:         try {
43:             MBeanServer mbs =
44:                 ManagementFactory.getPlatformMBeanServer();
45:             mbs.unregisterMBean(new
46:                 ObjectName("ru.astrosoup.weblab3:type=MissedHitCalculator"));
47:         } catch (Exception e) {
48:             log.severe(">>> MissedHitCalculator JMX unregistration
49: failed: " + e.getMessage());
50:         }
51:     }
52: }
```

```

45:
46:     @Override
47:     @Lock(LockType.READ)
48:     public double getMissedToTotalHitsInPercentsAllUsers() {
49:         long total = hitChecker.getTotalHitsAllUsers();
50:         if (total == 0) return 0.0;
51:         return (double) hitChecker.getMissedHitsAllUsers() / total *
100;
52:     }
53:
54:     @Override
55:     @Lock(LockType.READ)
56:     public double getMissedToTotalHitsInPercents(Long id) {
57:         long total = hitChecker.getTotalHits(id);
58:         if (total == 0) return 0.0; // also fix the division by zero
you currently have
59:         return (double) hitChecker.getMissedHits(id) / total * 100;
60:     }
61: }

```

Листинг 4 — Имплементация MissedHitCalculatorMBean

3. Показания JConsole

Для снятия показаний JConsole используем команду `jconsole -J-Djava.class.path=$JAVA_HOME/lib/jconsole.jar:./jboss-client.jar` для запуска. Нам нужен `jboss-client.jar` так как сервер WildFly не поддерживает удалённое подключение по JMX без него. Далее подключаемся к сервису при помощи `service:jmx:remote+http://localhost:9990`, выбираем MBeans и видим наши MBean'ы, а также их атрибуты и операции.

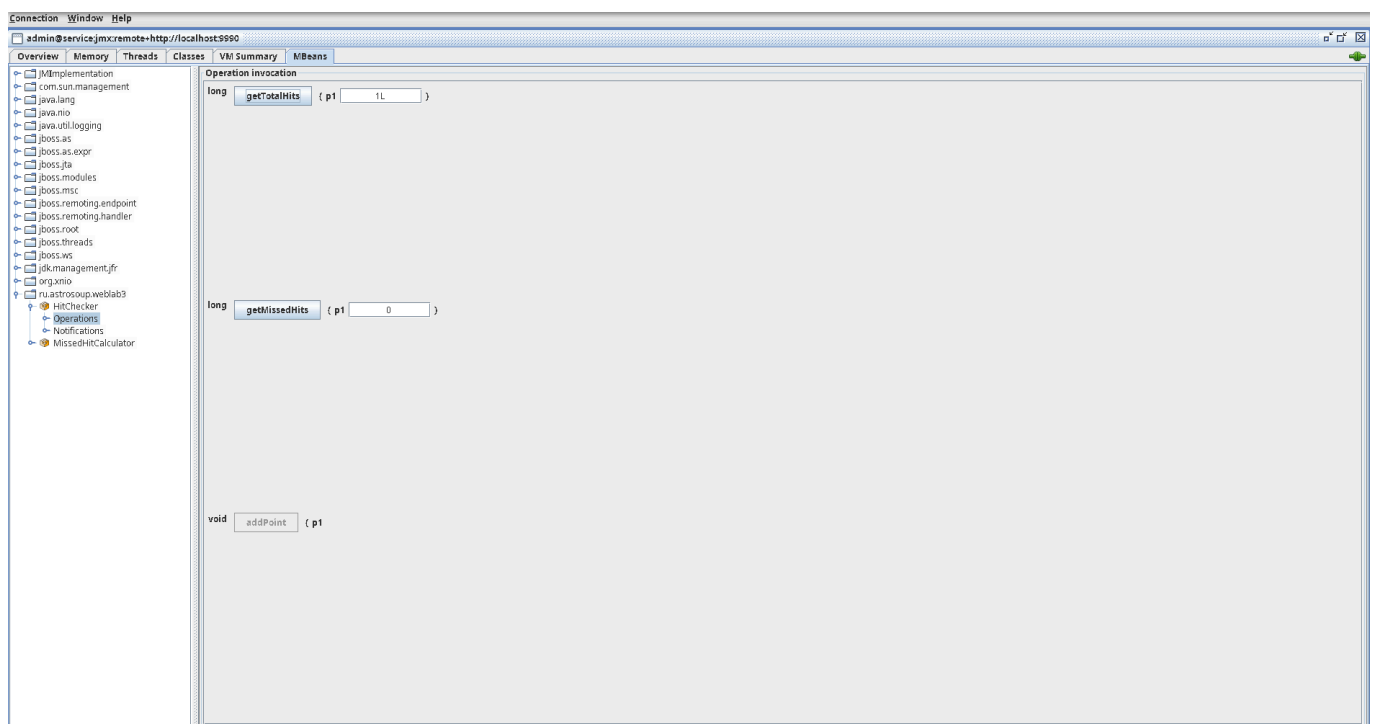


Рис. 1 — Страница MBeans и пользовательские Mbean'ы

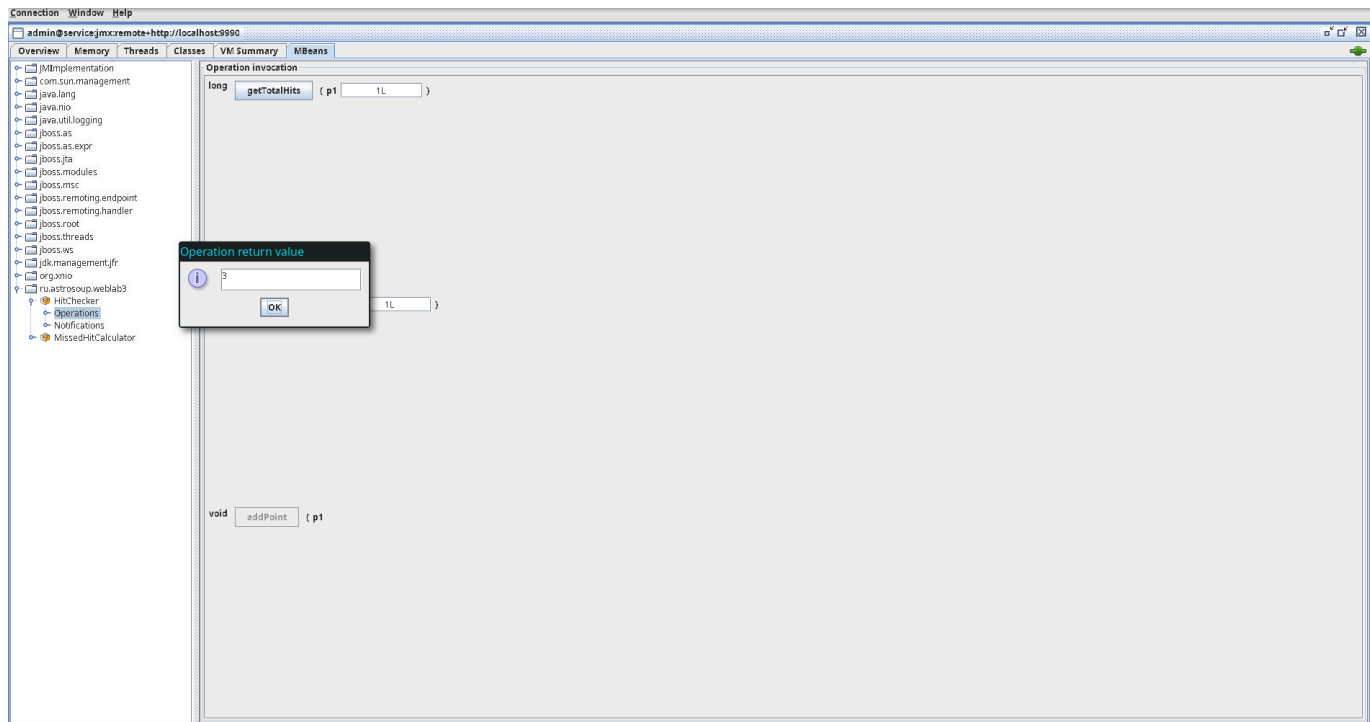


Рис. 2 — Показания вызванного метода

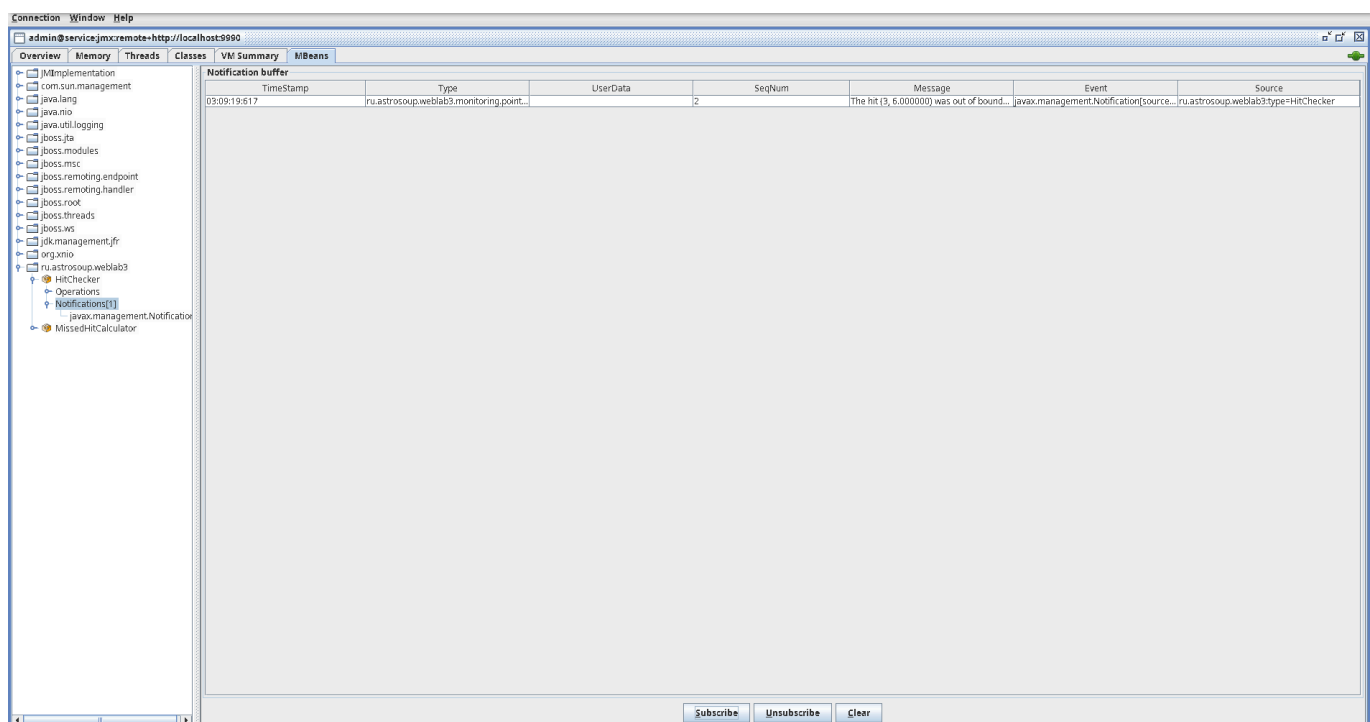


Рис. 3 — Уведомление от HitChecker

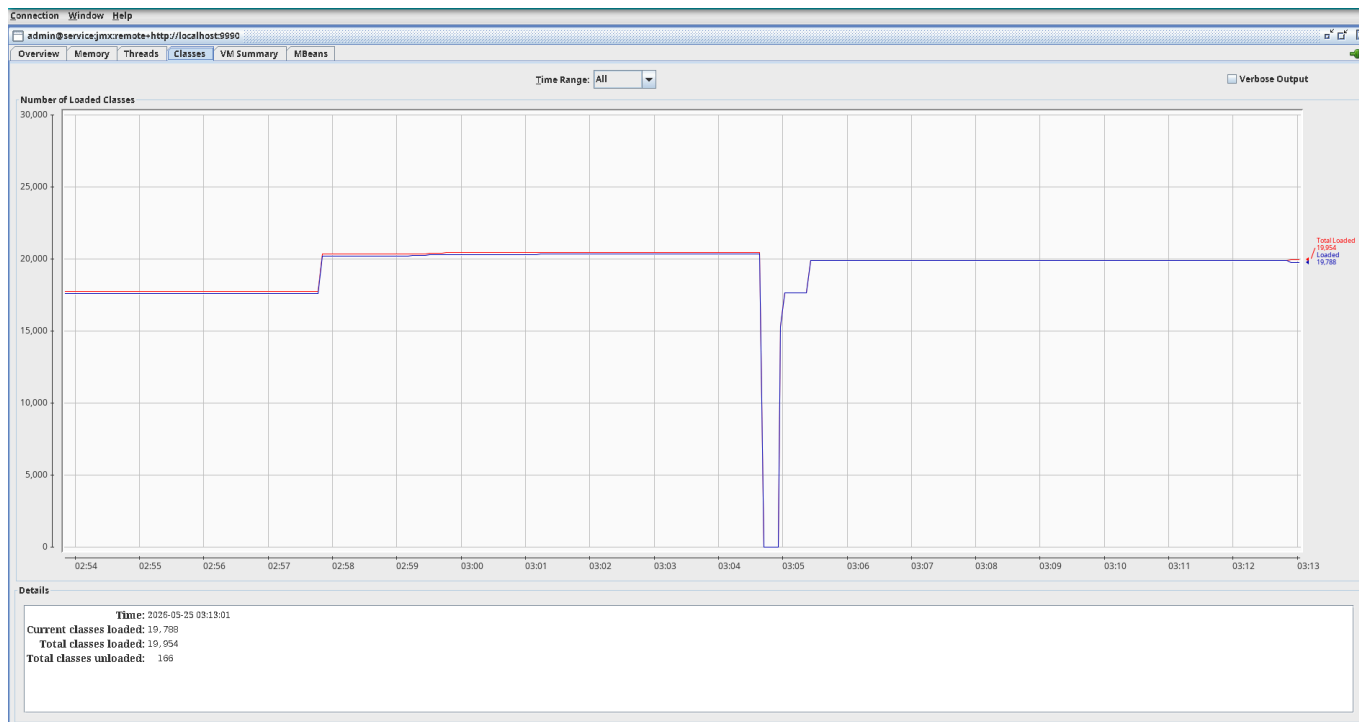


Рис. 4 — Мониторинг загруженных классов

4. Показания VisualVM

Мы также должны передать в classpath visualVM `jboss-client.jar`, чтобы подключаться к WildFly.

Для того чтобы VisualVM отображал атрибуты MBean'ов, необходимо чтобы методы доступа к ним не имели аргументов. Введем дополнительные агрегированные метрики по всем пользователям.

Графики показаний MBean'ов:

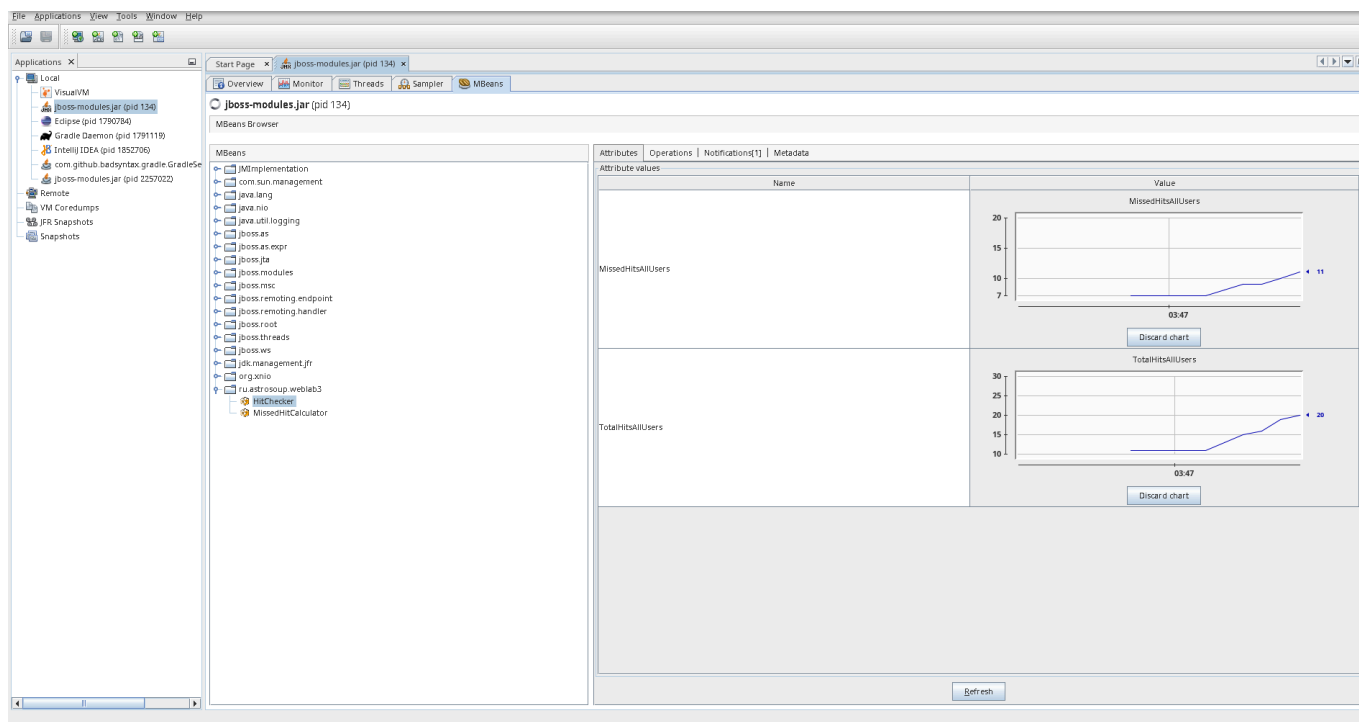


Рис. 5 — График для HitCheckerMBean

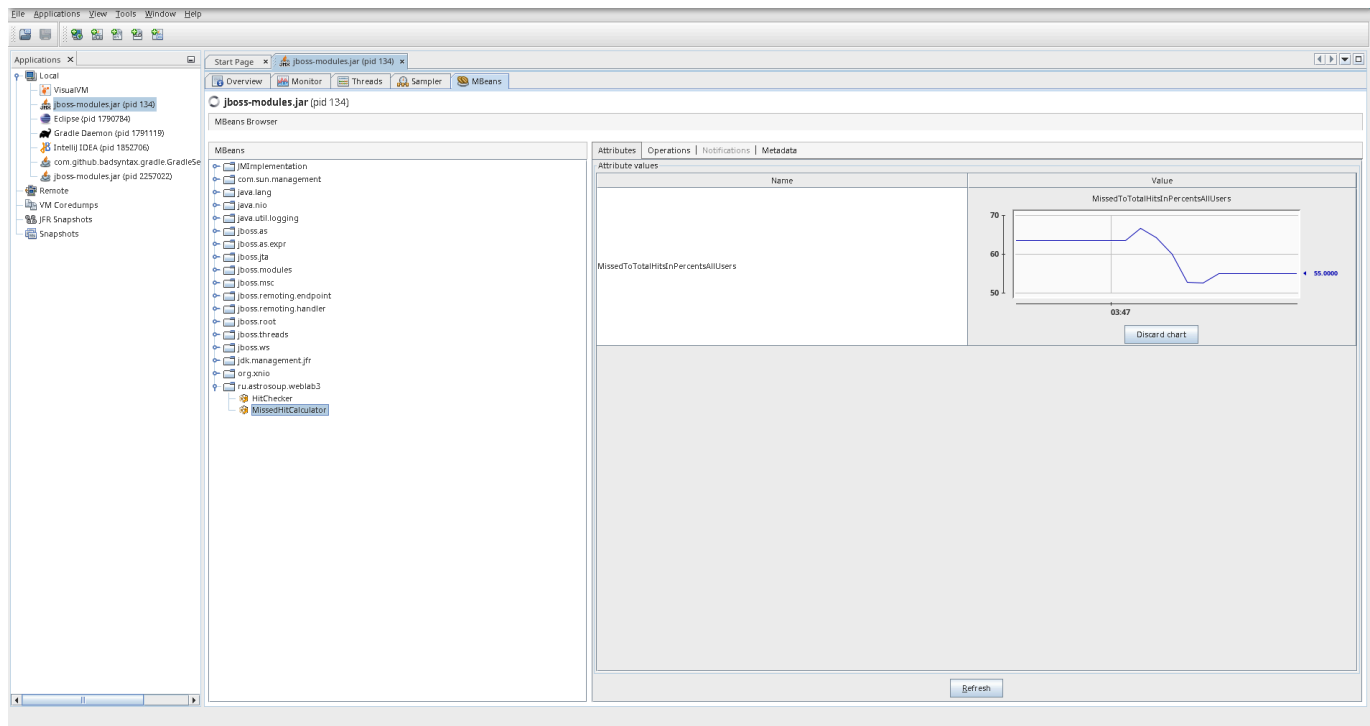


Рис. 6 — График для MissedHitCalculatorMBean

Чтобы найти класс занимающий наибольшее количество памяти в куче, перейдем в **Sampler -> Memory**, и найдем в поиске пользовательский модуль (в данном случае `ru.astrosoup.weblab3`), и увидим, что наибольшую память занимает класс `HitEntity`:

The screenshot shows the JBoss Modules Sampler Memory view. The 'Sample' dropdown is set to 'Memory'. The 'Status' is 'memory sampling in progress'. Below the status bar, there are tabs for 'Heap histogram' and 'Per thread allocations'. The main table lists various classes and their memory usage. The class 'ru.astrosoup.weblab3.entities.HitEntity' is highlighted in blue, indicating it has the highest memory usage.

Name	Live Bytes	Live Objects	Perform GC	Heap Dump
com.sun.management.VMOptions\$Origin	48 B	1	(0%)	
sun.nio.ch.IOVecWrappers\$Deallocator	48 B	3	(0%)	
org.jboss.logmanager.handlers.SyslogHandler\$SyslogType	48 B	2	(0%)	
org.jboss.as.server.BootstrapListener	48 B	1	(0%)	
org.jboss.as.controller.operations.global.ReadAttributeGroupHandler\$1	48 B	2	(0%)	
org.jboss.resteasy.core.QueryDecoder\$ClassActivator	48 B	3	(0%)	
org.jboss.as.server.security.VirtualDomainMetadata\$AuthMethod	48 B	2	(0%)	
org.jboss.as.controller.access.management.WritableAuthorizer\$Configurations\$MatchType	48 B	2	(0%)	
org.wildfly.extension.elytron.AbstractCredentialsStoreResourceDefinition\$CredentialsStoreRuntimeHandler	48 B	2	(0%)	
io.undertow.security.api.AuthenticationMode	48 B	2	(0%)	
org.wildfly.security.http.HttpAuthenticator\$1	48 B	2	(0%)	
org.jboss.dmr.BookmarkMode\$Value	48 B	2	(0%)	
io.undertow.server.protocol.ParseTimeoutUpdater\$1	48 B	3	(0%)	
org.jboss.domain.management.access.PrincipalResourceDefinition\$Type	48 B	2	(0%)	
com.fasterxml.jackson.core.StreamReadConstraints	48 B	1	(0%)	
ru.astrosoup.weblab3.entities.HitEntity	48 B	1	(0%)	
org.jboss.remoting.management.ManagementChannelOpenListenerService	48 B	1	(0%)	
java.lang.NullPointerException	48 B	1	(0%)	
org.xnio.XnioWorker\$1	48 B	3	(0%)	
org.wildfly.extension.elytron.ModifiableKeyStoreDecorators\$ReadAliasesHandler	48 B	3	(0%)	
com.sun.org.apache.xerces.internal.jaxp.validation.XMLSchemaFactory\$XMLGrammarPool\$ImplExtension	48 B	2	(0%)	
jakarta.json.JsonValue\$ValueType	48 B	1	(0%)	
java.time.ZonedDateTime	48 B	2	(0%)	
org.jboss.as.connector.subsystems.resourceadapters.WorkManagerRuntimeAttributeReadHandler	48 B	2	(0%)	
org.jboss.as.pojo.JsonAttributeListMarshaller	48 B	3	(0%)	
jakarta.persistence.FetchType	48 B	2	(0%)	
io.undertow.server.handlers.error.SimpleErrorHandler	48 B	2	(0%)	
io.ebeaninternal.server.core.CacheOptions	48 B	2	(0%)	
org.jboss.as.controller.operations.global.ReadResourceHandler\$Validator	48 B	2	(0%)	
org.jboss.remoting3.remote.RemoteConnection\$RemoteWriteListener	48 B	1	(0%)	
org.bithorn.validator.internal.annotation.validation.constraint.ValidateOnScreenOfContent	48 B	1	(0%)	

Рис. 7 — Пользовательский класс, занимающий наибольшую память в куче

5. Устранение проблем с производительностью в заданной программе

Запустив программу, и открыв ее процесс в visualVM, можем заметить, что куча заполняется, а сборщик мусора не может освободить память.

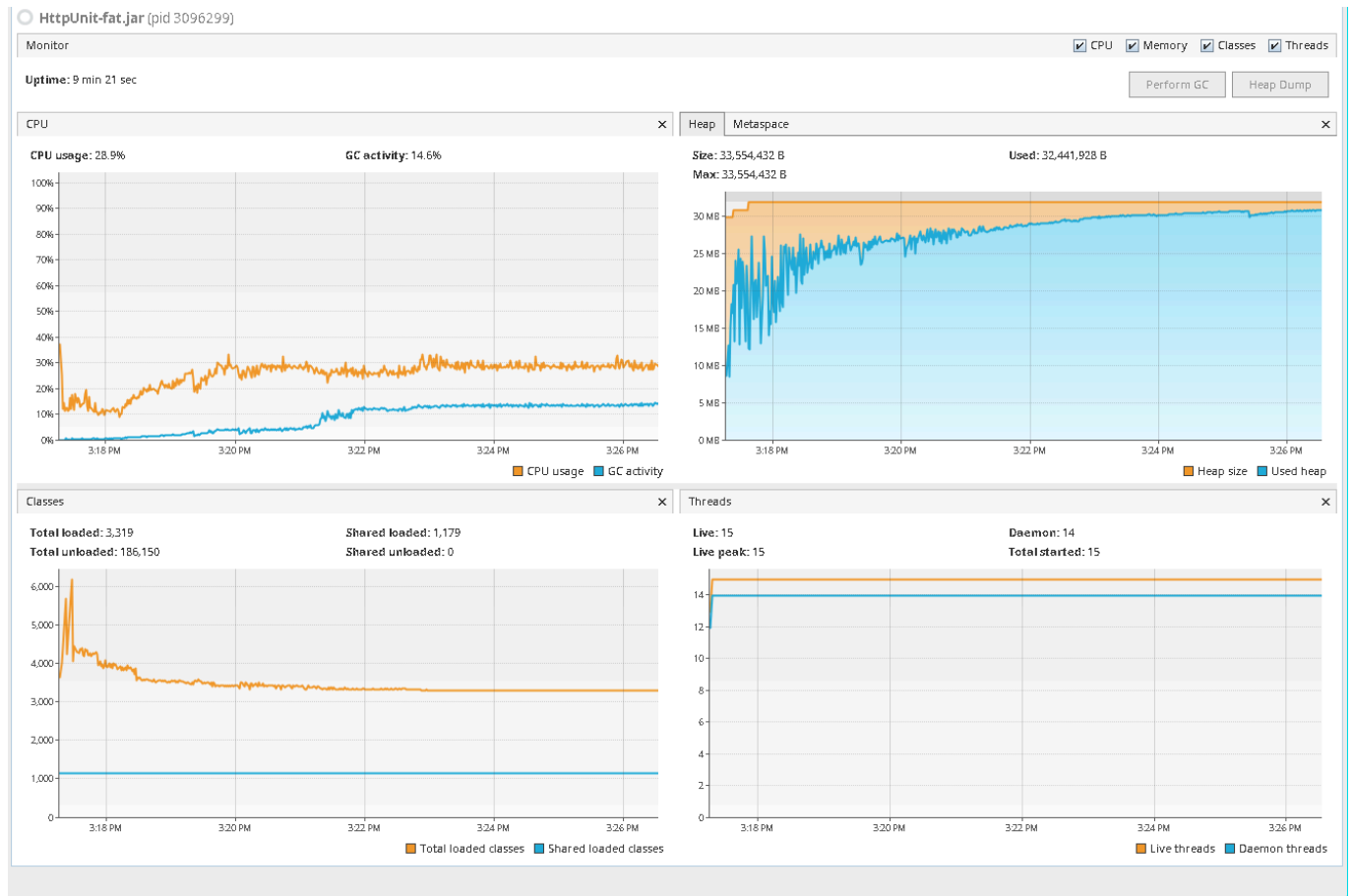


Рис. 8 — Показания VisualVM при запуске программы

Такое поведение свидетельствует об утечке памяти.

Уменьшим кучу до 32 мб и запустим программу, чтобы вызвать `OutOfMemoryError` и подтвердить утечку:

```
1: Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
2:   at java.base/
   java.util.concurrent.ConcurrentHashMap.transfer(ConcurrentHashMap.java:2507)
3:   at java.base/
   java.util.concurrent.ConcurrentHashMap.addCount(ConcurrentHashMap.java:2354)
4:   at java.base/
   java.util.concurrent.ConcurrentHashMap.putVal(ConcurrentHashMap.java:1075)
5:   at java.base/
   java.util.concurrent.ConcurrentHashMap.put(ConcurrentHashMap.java:1006)
6:   at java.base/java.util.Properties.put(Properties.java:1348)
7:   at java.base/java.util.Properties.load0(Properties.java:460)
8:   at java.base/java.util.Properties.load(Properties.java:385)
9:   at java.base/
   java.util.PropertyResourceBundle.<init>(PropertyResourceBundle.java:192)
```

```

10:   at java.base/
    java.util.PropertyResourceBundle.<init>(PropertyResourceBundle.java:169)
11:   at java.base/
    java.util.ResourceBundle$Control.newBundle0(ResourceBundle.java:3259)
12:   at java.base/
    java.util.ResourceBundle$Control.newBundle(ResourceBundle.java:3165)
13:   at java.base/
    java.util.ResourceBundle.loadBundle(ResourceBundle.java:1998)
14:   at java.base/
    java.util.ResourceBundle.findBundle(ResourceBundle.java:1784)
15:   at java.base/
    java.util.ResourceBundle.findBundle(ResourceBundle.java:1736)
16:   at java.base/
    java.util.ResourceBundle.findBundle(ResourceBundle.java:1736)
17:   at java.base/
    java.util.ResourceBundle.getBundleImpl(ResourceBundle.java:1670)
18:   at java.base/
    java.util.ResourceBundle.getBundleImpl(ResourceBundle.java:1600)
19:   at java.base/
    java.util.ResourceBundle.getBundleImpl(ResourceBundle.java:1555)
20:   at java.base/
    java.util.ResourceBundle.getBundle(ResourceBundle.java:935)
21:   at org.mozilla.javascript.Context.getMessage(Context.java:1945)
22:   at org.mozilla.javascript.Context.getMessage1(Context.java:1907)
23:   at
    org.mozilla.javascript.ScriptRuntime.getMessage1(ScriptRuntime.java:1977)
24:   at
    org.mozilla.javascript.NativeGlobal.typeError1(NativeGlobal.java:567)
25:   at org.mozilla.javascript.ScriptRuntime.call(ScriptRuntime.java:1182)
26:   at org.mozilla.javascript.gen.c107929.call(httpunit:0)
27:   at org.mozilla.javascript.gen.c107929.exec(httpunit)
28:   at org.mozilla.javascript.Context.evaluateReader(Context.java:820)
29:   at org.mozilla.javascript.Context.evaluateString(Context.java:784)
30:   at
    com.meterware.httpunit.javascript.JavaScript$JavaScriptEngine.executeScript(Ja
31:   at
    com.meterware.httpunit.scripting.ScriptableDelegate.runScript(ScriptableDelega
32:   at
    com.meterware.httpunit.parsing.ScriptFilter.getTranslatedScript(ScriptFilter.j
33:   at
    com.meterware.httpunit.parsing.ScriptFilter.endElement(ScriptFilter.java:131)

```

из дампа кучи можем заметить что в классе JavaScript есть статическое поле `_errorMessages`, в котором находится подозрительно много объектов.

java.lang.Object[]	20,506	(3.6%)	976,520 B	(4%)	n/a
java.lang.Object[]#19234 : 71,140 items			284,576 B	(1.2%)	n/a
<items>					
<references>					
elementData in java.util.ArrayList#14 : 57,281 elements			24 B	(0%)	n/a
static _errorMessages in class com.meterware.httpunit.javascript.JavaScript : JavaScript			72 B	(0%)	n/a
[51] in java.lang.Object[]#648 : 474 items			1,912 B	(0%)	n/a
elementData in java.util.ArrayList#1072 : 435 elements			24 B	(0%)	n/a

Рис. 9 — Интересующее нас поле в дампе кучи

Более подробно рассмотрев класс заметим что это поле никогда не очищается, и при большом количестве ошибок может привести к заполнению кучи. Решением проблемы будет очистка после каждого вызова.

После добавления очистки в основной цикл Main класса, мы можем наблюдать нормальное поведение кучи, и отсутствие `OutOfMemoryError`:

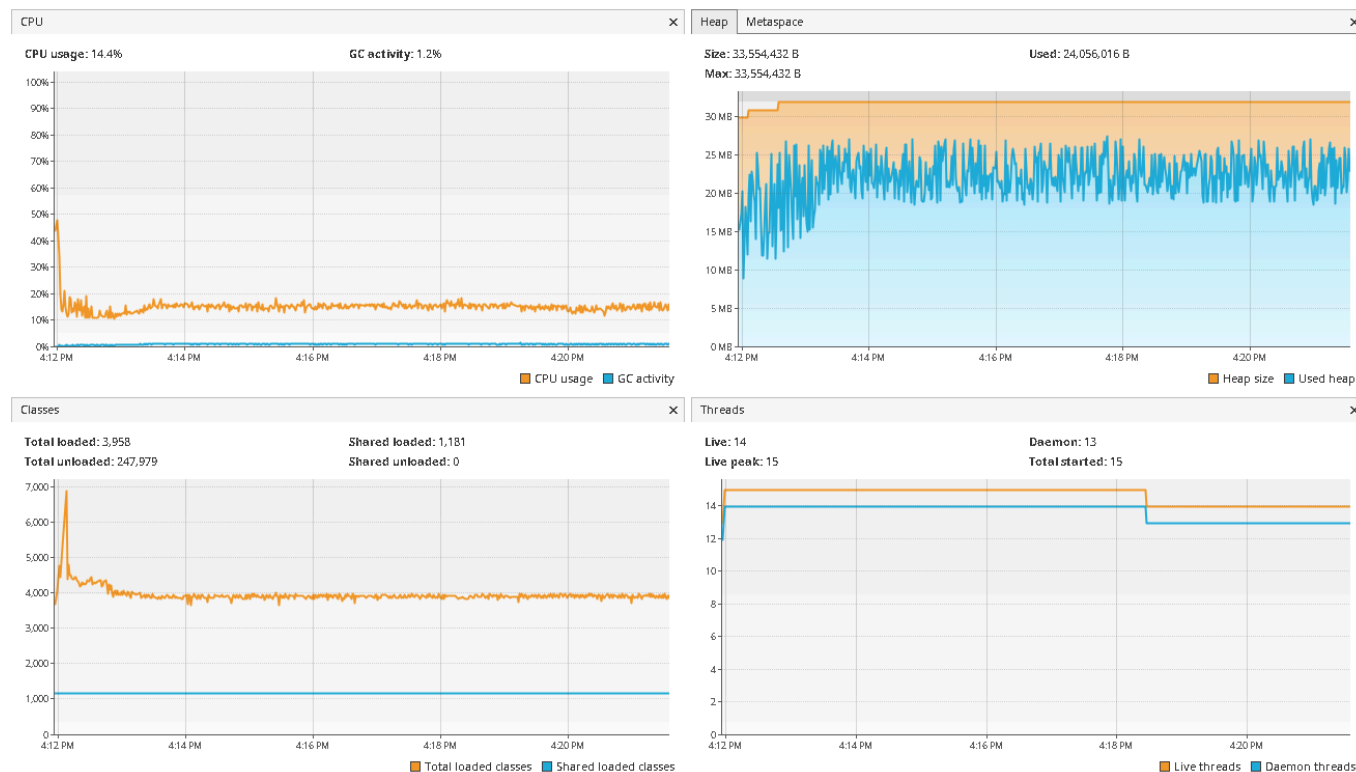


Рис. 10 — Показания после исправления

6. Вывод

В ходе выполнения лабораторной работы я познакомился с инструментами мониторинга JConsole и VisualVM, написал Mbean'ы для мониторинга и устранил утечку памяти в предложенной программе.